

How to rewrite world to Rust?

Starting from e-radio mp3 decoding.

Dawid Królak

- Expert engineer @ mobica a cognizant company
- Passionate about space exploration industry

<https://twitter.com/Taavicjusz>

<https://github.com/taavit>

<https://mastodon.social/@taavit>

Agenda

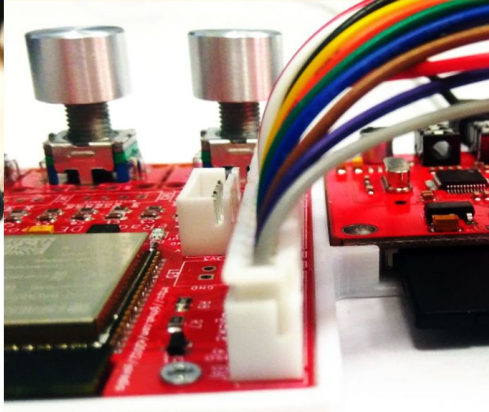
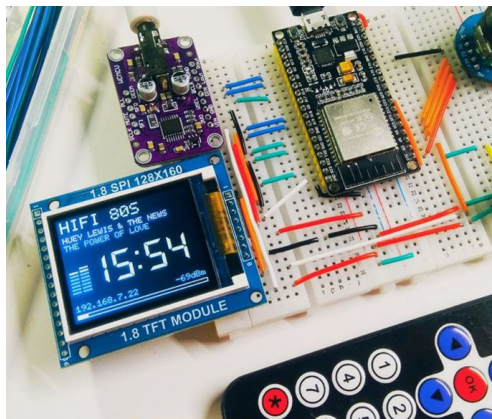
- Why yoRadio
- Rust Integration
- Challenges
- Charts
- Conclusions

What is Rust

- System programming language
- Single ownership for variables
- Type safety
- Lifetime for references
- Separation of safe and unsafe code
- Unified ecosystem (cargo)
 - Testing methods
 - Dependency management
 - Built in linter and formatter

Why yoRadio

- Hey, I've seen nice e-radio, I want one!
- I don't know what it is...
- Check this facebook group!



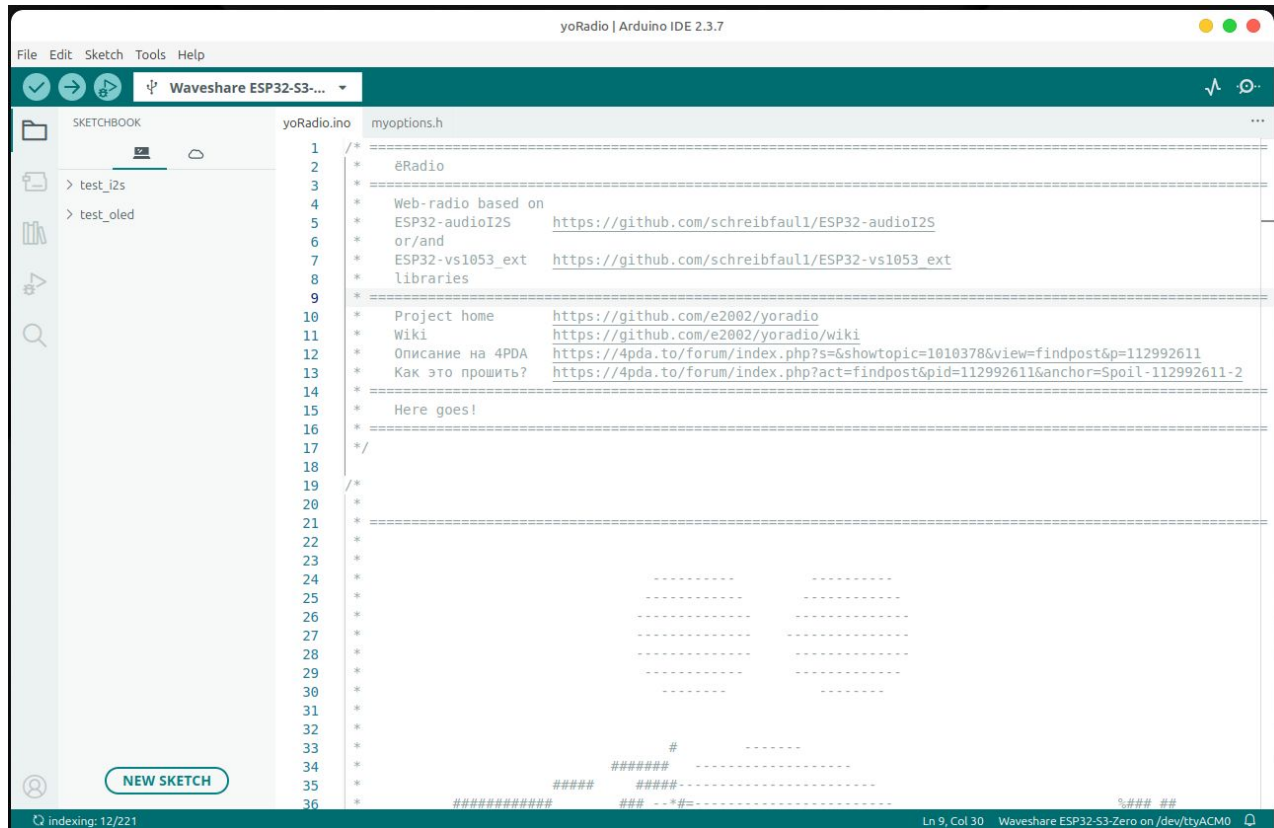
yoRadio

Arduino IDE	Nope
C/C++ Arduino framework	Nope
I ² S protocol	Nope
ESP32	Nope

Verdict: **GO!**

Used hardware

- 1x Waveshare ESP32-S3 Mini
- 1x PCM5100 I²S DAC
- 1x OLED 0,96" 128x64 I²C
- 3x Tact switch (encoder in later version)
- 1x Breadboard
- Nx Jumper wires



What if?

Rust integration

- Arduino IDE - No Rust support
- Rustup - No arduino support
- VS Code - No Arduino support...

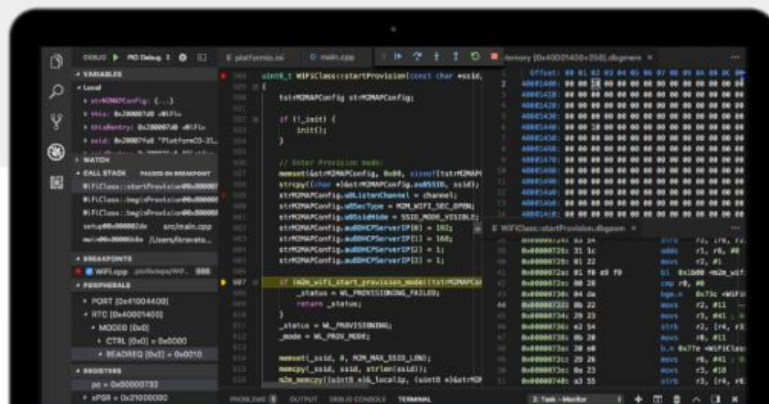
[Get Started](#)[Solutions](#)[Registry](#)[Docs](#)[Community](#)[Support](#)[TECHNOLOGY](#)

PlatformIO IDE

A new generation toolset for embedded C/C++ development

[Get for VSCode](#)[Looking for another editor?](#)

It's built on top of [Microsoft's Visual Studio Code](#) – free, open source, and MIT licensed editor



PlatformIO is a Ukrainian Project



Please help us stop the war.

[STOP WAR](#)

Plan

- Compile pure yoRadio in platformio
- Build dummy static library for xtensa
- Replace algorithmic part with rust drop in replacement

platformio.ini

```
[env:esp32-s3-zero]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
board_upload.flash_size = 4MB
board_build.partitions = min_spiffs.csv
build_flags =
    -DARDUINO_USB_CDC_ON_BOOT=1
    -DBOARD_HAS_PSRAM
    -Llib
    -lrust_mod

lib_deps =
    adafruit/Adafruit GFX Library@^1.12.1
    adafruit/Adafruit SSD1306@^2.5.15
```

Rust for Espressif

» Xtensa Devices

As mentioned in [hardware overview](#) ESP32, ESP32-S2 and ESP32-S3 are based on Xtensa architecture. If you are going to target these chips you will need to use a fork of the Rust compiler for now.

`espup` is a tool that simplifies installing and maintaining the toolchains required to develop Rust applications for these targets.

1. Install `espup` :

```
cargo install espup --locked
```

You can also directly download pre-compiled [release binaries](#) or use `cargo-bininstall`.

2. Install all necessary toolchains for all supported Espressif targets by running:

```
espup install
```

3. On Unix systems, set up the Environment Variables: See the different methods in [espup README](#). Windows users don't need to do anything else.

Dummy static library

lib.rs

```
#![no_std]
use core::panic::PanicInfo;

#[unsafe(no_mangle)]
pub extern "C" fn rust_add(left: i32, right: i32)
-> i32 {
    left + right
}

#[panic_handler]
fn panic_handler(_: &PanicInfo) -> ! {
    loop {}
}
```

Cargo.toml

```
[package]
name = "rust_mod"
version = "0.1.0"
edition = "2024"

[lib]
crate-type = ["staticlib"]

[dependencies]
[profile.release]
opt-level = "s"
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true
debug = false
overflow-checks = false
incremental = false
```

Dummy static library (rust_mod.h)

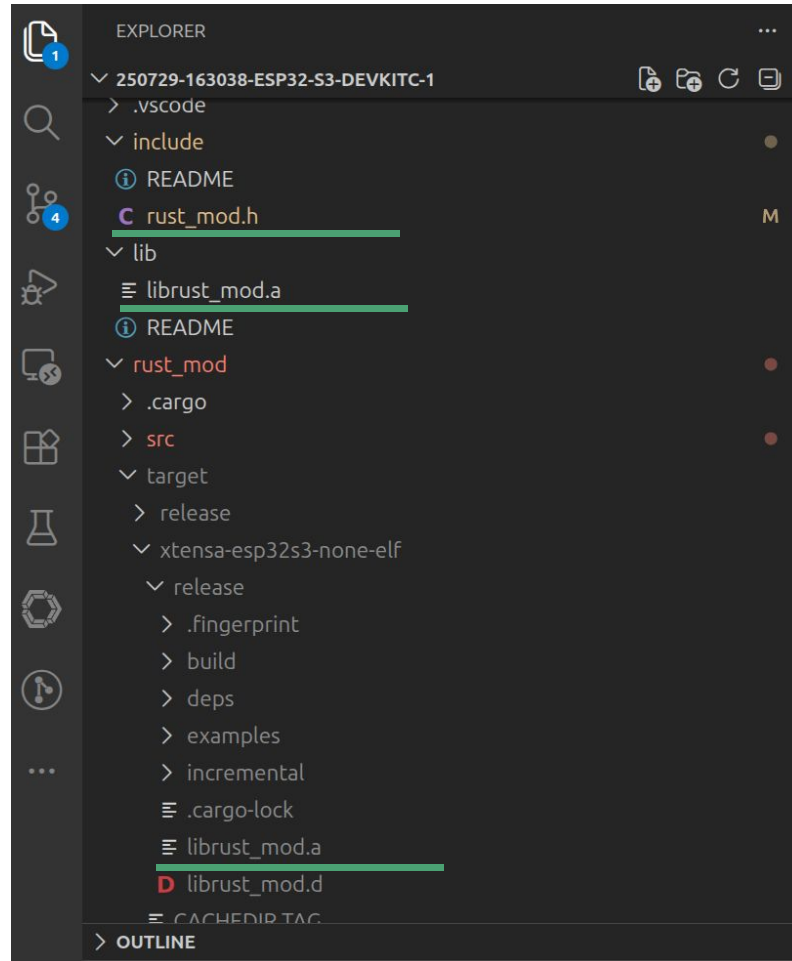
```
#pragma once

#ifdef __cplusplus
extern "C" {
#endif

int32_t rust_add(int32_t a, int32_t b);

#ifdef __cplusplus
}
#endif
```

Final structure



Dummy static library

```
#include "rust_mod.h"

void MyNetwork::raiseSoftAP() {
    WiFi.mode(WIFI_AP);
    WiFi.softAP(apSsid, apPassword);
    Serial.println("##[BOOT]#");
    BOOTLOG("*****");
    BOOTLOG("Running in AP mode");
    BOOTLOG("Connect to AP %s with password %s", apSsid, apPassword);
    BOOTLOG("and go to http://192.168.4.1/ to configure");
    BOOTLOG("*****");
    BOOTLOG("NUMBERS FROM RUST = %i", rust_add(10, 32));
    status = SOFT_AP;
    if(config.store.softapdelay>0)
        timekeeper.waitAndDo(config.store.softapdelay*60, rebootTime);
}
```

Let's start with real code!

Choosing code

- No hardware related - focus on algorithms
- No c++ class - no deal c++ mangling during process
- Pure process - data in, data out

Final form

- Single unsafe call - entry mp3decode function
- Idiomatic Rust - no pointers, iterators, enums etc
- Acceptable performance
- Maybe some tests

Challenges

Macros

```
/* clip to range  $[-2^n, 2^n - 1]$  */
#if 0 //Fast on ARM:
#define CLIP_2N(y, n) { \
    int sign = (y) >> 31; \
    if (sign != (y) >> (n)) { \
        (y) = sign ^ ((1 << (n)) - 1); \
    } \
}
#else //on xtensa this is faster, due to asm
min/max instructions:
#define CLIP_2N(y, n) { \
    int x = 1 << n; \
    if (y < -x) y = -x; \
    x--; \
    if (y > x) y = x; \
}
#endif
```

```
macro_rules! clip_2n {
    ($y:expr, $n:expr) => {{
        let bound = 1i32 << n;
        *y = (*y).min(bound - 1).max(-bound)
    }};
}

#[inline(always)]
#[cfg(target_arch = "xtensa")]
fn CLIP_2N(y: &mut i32, n: i32) {
    let bound = 1i32 << n;
    *y = (*y).min(bound - 1).max(-bound)
}
```

Pointers

```
*xy++ = y;

cache |= (unsigned int) (*buf++) << (24 - cachedBits);
cw = pgm_read_word(&tBase[cache >> (32 - maxBits)]);

while (cachedBits < minBits) {
    cache |= (unsigned int) (*buf++) << (24 - cachedBits);
    cachedBits += 8;
    bitsLeft -= 8;
}

memcpy(m_MP3DecInfo->mainBuf + m_MP3DecInfo->mainDataBytes, inbuf, m_MP3DecInfo->nSlots);
m_MP3DecInfo->mainDataBytes += m_MP3DecInfo->nSlots;
inbuf += m_MP3DecInfo->nSlots;
*bytesLeft -= (m_MP3DecInfo->nSlots);
MP3ClearBadFrame( outbuf);
return ERR_MP3_MAINDATA_UNDERFLOW;
```

Pointers

```
for i in 0..9 {  
    *y = - *y;  
    y = y.add(2 * m_NBANDS as usize);  
}  
  
let y: &mut [i32; m_NBANDS as usize * 18] = unsafe { &mut *(y as *mut [i32; 18 * m_NBANDS as usize]) };  
  
// memcpy(mainBuf + mainDataBytes, inbuf, nSlots)  
core::ptr::copy_nonoverlapping(  
    inbuf,  
    (*m_MP3DecInfo).mainBuf.as_mut_ptr().add((*m_MP3DecInfo).mainDataBytes as usize),  
    (*m_MP3DecInfo).nSlots as usize  
);
```

Slices

```
let cptr1: &[i32; 24] = &cptr_all[1];
let (chunks, _) = buf_slice.as_chunks_mut::<8>();
let (cptr_chunks, _) = cptr1.as_chunks::<6>();

for (buf_chunk, cptr) in chunks.iter_mut().zip(cptr_chunks) {
    let b0 = buf_chunk[0] + buf_chunk[7];
    let b7 = mulshift_32(cptr[0], buf_chunk[0] - buf_chunk[7]) << 1;
    let b3 = buf_chunk[3] + buf_chunk[4];
    let b4 = mulshift_32(cptr[1], buf_chunk[3] - buf_chunk[4]) << 3;
}
```

What about bound check?

```
#[unsafe(no_mangle)]
pub fn slice_1(a: &[i32; 4]) -> i32 {
    let mut sum = 0;
    for i in 0..4 {
        sum += a[i];
    }
    sum
}
```

```
slice_1:
    lw      a1, 0(a0)
    lw      a2, 4(a0)
    lw      a3, 8(a0)
    lw      a0, 12(a0)
    add     a1, a2, a1
    add     a0, a0, a3
    add     a0, a0, a1
    ret
```

```
rustc 1.94 -C opt-level=3 --target=riscv32i-unknown-none-elf
```

What about bound check?

```
#[unsafe(no_mangle)]
pub fn slice_2(a: &[i32]) -> i32 {
    let mut sum = 0;
    for i in 0..4 {
        sum += a[i];
    }
    sum
}
```

```
slice_2:
    li      a2, 4
    bltu    a1, a2, .LBB0_2
    lw      a1, 0(a0)
    lw      a2, 4(a0)
    lw      a3, 8(a0)
    lw      a0, 12(a0)
    add     a1, a2, a1
    add     a0, a0, a3
    add     a0, a0, a1
    ret

.LBB0_2:
    lui     a2, %hi(.Lanon.21d56e13bad70e8f9749353087ce44b3 .1)
    addi    a2, a2, %lo(.Lanon.21d56e13bad70e8f9749353087ce44b3 .1)
    mv      a0, a1
    call    core::panicking::panic_bounds_check::hd5db10c9f90963bb

.Lanon.21d56e13bad70e8f9749353087ce44b3 .0:
    .asciz  "/app/example.rs"

.Lanon.21d56e13bad70e8f9749353087ce44b3 .1:
    .word   .Lanon.21d56e13bad70e8f9749353087ce44b3 .0
    .asciz  "\017\000\000\000\020\000\000\000\020\000\000"
```

```
rustc 1.94 -C opt-level=3 --target=riscv32i-unknown-none-elf
```

What about bound check?

```
#[unsafe(no_mangle)]
pub fn slice_3(a: &[i32; 4]) -> i32
{
    let mut sum = 0;
    for i in 0..8 {
        sum += a[i];
    }
    sum
}
```

```
slice_3:
    lui     a2, %hi(.Lanon.5e5d7260bc7257a3479b708a085a1e2f .1)
    addi    a2, a2, %lo(.Lanon.5e5d7260bc7257a3479b708a085a1e2f .1)
    li      a0, 4
    li      a1, 4
    call    core::panicking::panic_bounds_check::hd5db10c9f90963bb

.Lanon.5e5d7260bc7257a3479b708a085a1e2f .0:
    .asciz  "/app/example.rs"

.Lanon.5e5d7260bc7257a3479b708a085a1e2f .1:
    .word   .Lanon.5e5d7260bc7257a3479b708a085a1e2f .0
    .asciz  "\017\000\000\000\031\000\000\000\020\000\000"
```

```
rustc 1.94 -C opt-level=3 --target=riscv32i-unknown-none-elf
```

What about bound check?

```
#[unsafe(no_mangle)]
pub fn slice_4(a: &[i32; 4]) -> i32
{
    let mut sum = 0;
    for el in a {
        sum += el;
    }
    sum
}
```

```
slice_4:
    lw     a1, 0(a0)
    lw     a2, 4(a0)
    lw     a3, 8(a0)
    lw     a0, 12(a0)
    add    a1, a2, a1
    add    a0, a0, a3
    add    a0, a0, a1
    ret
```

```
rustc 1.94 -C opt-level=3 --target=riscv32i-unknown-none-elf
```

What about bound check?

```
#[unsafe(no_mangle)]
pub fn slice_5(a: &[i32]) -> i32 {
    if a.len() < 4 {
        return 0;
    }
    let mut sum = 0;
    for i in 0..4 {
        sum += a[i];
    }
    sum
}
```

```
slice_5:
    li      a2, 4
    bgeu    a1, a2, .LBB0_2
    li      a0, 0
    ret

.LBB0_2:
    lw      a1, 0(a0)
    lw      a2, 4(a0)
    lw      a3, 8(a0)
    lw      a0, 12(a0)
    add     a1, a2, a1
    add     a0, a0, a3
    add     a0, a0, a1
    ret
```

```
rustc 1.94 -C opt-level=3 --target=riscv32i-unknown-none-elf
```

Algorithms

```
for (cb = cbStartS; cb < cbEndS; cb++) {

    nSamps = m_SFBandTable.s[cb + 1] - m_SFBandTable.s[cb];
    for (w = 0; w < 3; w++) {
        nonZero = 0;
        gainI = 210 - globalGain + 8*sis->subBlockGain[w] + sfactMultiplier*(sfis->s[cb][w]);

        nonZero |= DequantBlock(sampleBuf + i + nSamps*w, workBuf + nSamps*w, nSamps, gainI);

        /* update highest non-zero critical band */
        if (nonZero)
            cbMax[w] = cb;
        gbMask |= nonZero;
    }

    /* reorder blocks */
    buf = (ARRAY3 *) (sampleBuf + i);
    i += 3*nSamps;
    for (j = 0; j < nSamps; j++) {
        buf[j][0] = workBuf[0*nSamps + j];
        buf[j][1] = workBuf[1*nSamps + j];
        buf[j][2] = workBuf[2*nSamps + j];
    }

    assert(3*nSamps <= m_MAX_REORDER_SAMPS);

    if (i >= *nonZeroBound)
        break;
}
```

Algorithms

```
for cb in cb_start_s..cb_end_s {
  let n_samps =
    (m_sf_band_table.s[(cb + 1) as usize] - m_sf_band_table.s[cb as
    usize]) as i32;

  for w in 0..3 {
    let gain_i = 210 - global_gain
      + 8 * sis.sub_block_gain[w]
      + s_multiplier * (sfis.s[cb as usize][w] as i32);

    // Dekwantyzujemy do workBuf, aby móc potem bezpiecznie przełożyć
    dane do sampleBuf
    let buf_offset = (n_samps * w as i32) as usize;
    let non_zero = dequant_block (
      &sample_buf[i + buf_offset as usize..i + buf_offset + n_samps as
      usize],
      &mut work_buf[buf_offset as usize..buf_offset + n_samps as
      usize],
      gain_i,
    );

    if non_zero != 0 {
      cb_max[w] = cb;
    }
    gb_mask |= non_zero;
  }
}
```

Yes, that's LLMs work.

```
// 4. Reordering: Przeplatanie próbek z 3 bloków krótkich
// C: buf[j][0] = workBuf[0*nSamps + j]
let n = n_samps as usize;
let start = i;
let end = start + n * 3;

// Bezpośrednie wycięcie fragmentu (rzuci panic jeśli poza zakresem)
let target_slice = &mut sample_buf[start..end];
let (rows, _) = target_slice.as_chunks_mut::<3>();

for (j, row) in rows.iter_mut().enumerate() {
  // row jest &mut [i32; 3], kompilator usuwa sprawdzenie granic dla
  row[0,1,2]
  row[0] = work_buf[j];
};

i += (3 * n_samps) as usize;
if i >= (*non_zero_bound) as usize {
  break;
}
}
```

MP3 helix decoder itself...

```
typedef struct BitStreamInfo {  
    unsigned char *bytePtr;  
    unsigned int iCache;  
    int cachedBits;  
    int nBytes;  
} BitStreamInfo_t;
```

```
void RefillBitstreamCache(BitStreamInfo_t *bsi) {  
    int nBytes = bsi->nBytes;  
    /* optimize for common case, independent of machine  
    endian-ness */  
    if (nBytes >= 4) {  
        bsi->iCache = (*bsi->bytePtr++) << 24;  
        bsi->iCache |= (*bsi->bytePtr++) << 16;  
        bsi->iCache |= (*bsi->bytePtr++) << 8;  
        bsi->iCache |= (*bsi->bytePtr++);  
        bsi->cachedBits = 32;  
        bsi->nBytes -= 4;  
    } else {  
        bsi->iCache = 0;  
        while (nBytes-- > 0) {  
            bsi->iCache |= (*bsi->bytePtr++) << (8 * nBytes);  
        }  
        bsi->iCache <<= ((3 - bsi->nBytes) * 8);  
        bsi->cachedBits = 8 * bsi->nBytes;  
        bsi->nBytes = 0;  
    }  
}
```

MP3 Helix decoder itself...

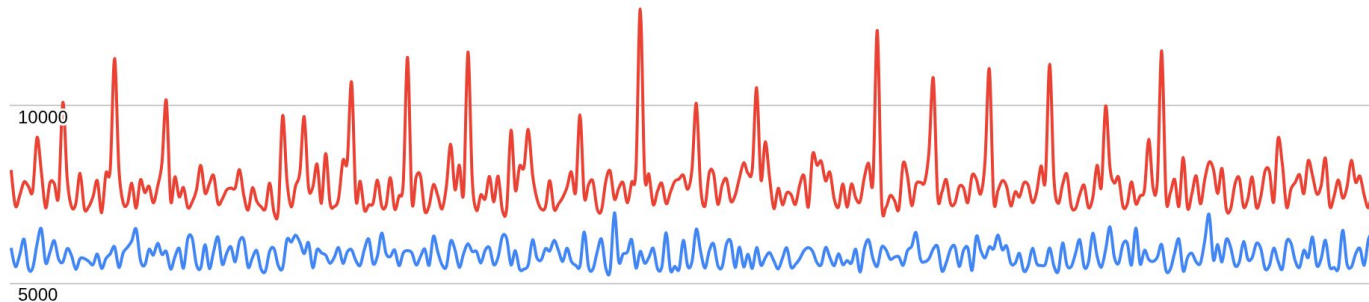
```
/* let c(j) = cos(M_PI/36 * ((j)+0.5)), s(j) = sin(M_PI/36 * ((j)+0.5))
* then fastWin[2*j+0] = c(j)*(s(j) + c(j)), j = [0, 8]
*      fastWin[2*j+1] = c(j)*(s(j) - c(j))
* format = Q30
*/
const uint32_t fastWin36[18] PROGMEM = {
    0x42aace8b, 0xc2e92724, 0x47311c28, 0xc95f619a, 0x4a868feb, 0xd0859d8c,
    0x4c913b51, 0xd8243ea0, 0x4d413ccc, 0xe0000000, 0x4c913b51, 0xe7dbc161,
    0x4a868feb, 0xef7a6275, 0x47311c28, 0xf6a09e67, 0x42aace8b, 0xfd16d8dd
};
```

Measurements...

After 2 months of fun with RIIR...

Optimized Rust vs Non optimized

— After optimization — Non optimized Rust (default)

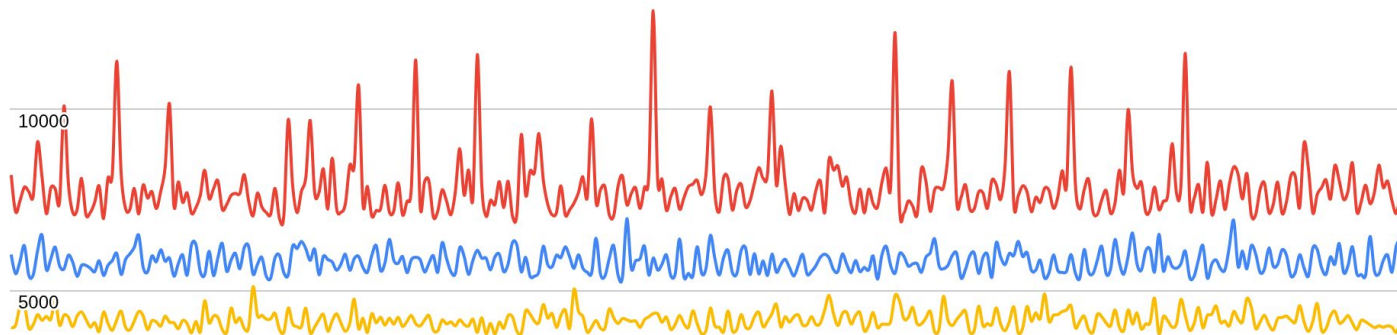


Stats	Min (us)	Max (us)	Average (us)	Median (us)	StdDev (us)
Non optimized Rust	6987	12717	7873,685619	7718	889,1998976
Optimized Rust	5317	6985	5803,772575	5785	311,1192856

After 2 months of work... Compared with C

Optimized Rust vs Non optimized

— After optimization — Non optimized Rust (default) — C

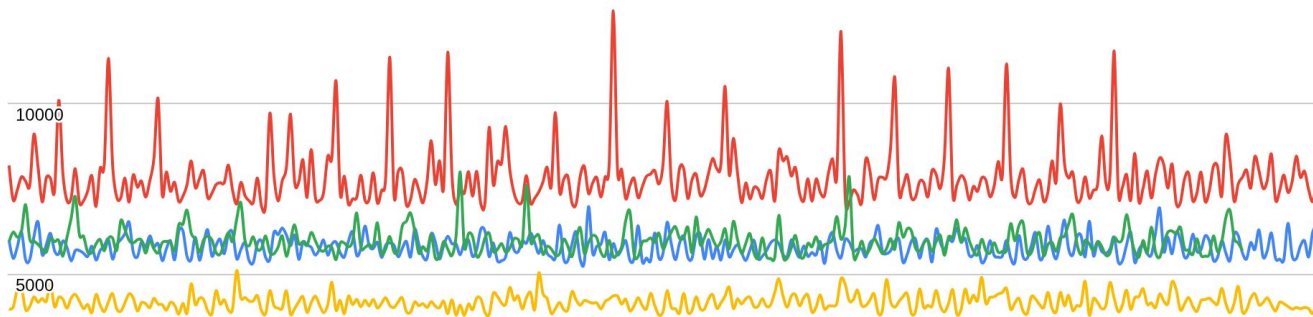


Stats	Min (us)	Max (us)	Average (us)	Median (us)	StdDev (us)
C	3798	5113	4192,879599	4154	240,4884408
Non optimized Rust	6987	12717	7873,685619	7718	889,1998976
Optimized Rust	5317	6985	5803,772575	5785	311,1192856

After 2 months of work... Initial with flags

Optimized Rust vs Non optimized

After optimization Non optimized Rust (default) C Non-optimized opt-level "s"

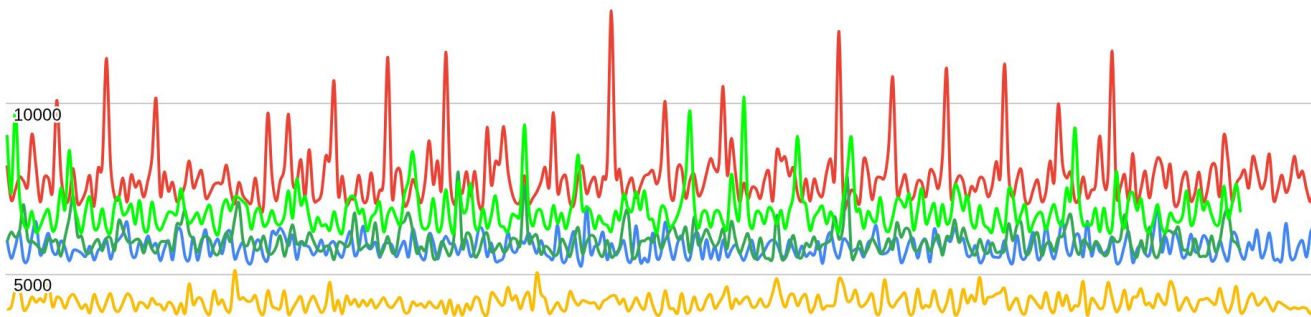


Stats	Min (us)	Max (us)	Average (us)	Median (us)	StdDev (us)
C	3798	5113	4192,879599	4154	240,4884408
Non optimized Rust	6987	12717	7873,685619	7718	889,1998976
Optimized Rust	5317	6985	5803,772575	5785	311,1192856
Non optimized Rust (with flags)	5430	8000	5976,173913	5939	382,5041429

And optimization without flags

Optimized Rust vs Non optimized

After optimization Non optimized Rust (default) C Non-optimized opt-level "s" Optimized without flags



Stats	Min (us)	Max (us)	Average (us)	Median (us)	StdDev (us)
C	3798	5113	4192,879599	4154	240,4884408
Non optimized Rust	6987	12717	7873,685619	7718	889,1998976
Optimized Rust	5317	6985	5803,772575	5785	311,1192856
Non optimized Rust (with flags)	5430	8000	5976,173913	5939	382,5041429
Optimized without flags	6169	10196	6877,468227	6804	609,3237049

What happened?

Tooling

- Lack of measurements from the start.
- Lack of understanding code I'm rewriting.
- Too strict requirement? Allow some unsafe?

Tooling

- Ghidra
- Godbolt
- `rustc -emit=asm`

Testing

- More testing from day 1
- Use platformio/rust testing environment directly on ESP32
- Run benchmarks directly on ESP32

Ok(Agenda::Questions)
